

AtmoKHI

Karachi AQI Prediction and Analytics System

Prepared by: Murtaza Aamir

Live Dashboard: atmokhi.streamlit.app

GitHub Repository: github.com/murtaza-23/AtmoKHI

1. Project Overview

AtmoKHI is an end-to-end machine learning system that predicts Karachi's Air Quality Index (AQI) up to 72 hours ahead. Built during the 10Pearls Shine Internship, it covers automated ingestion, feature storage, model training, explainability, and cloud deployment.

The system operates autonomously: hourly pipelines collect live pollutant readings, while daily workflows retrain models on recent data. An interactive web application presents real-time AQI, 3-day forecasts, model performance metrics, and public health advisories.

1.1 Architecture Diagram

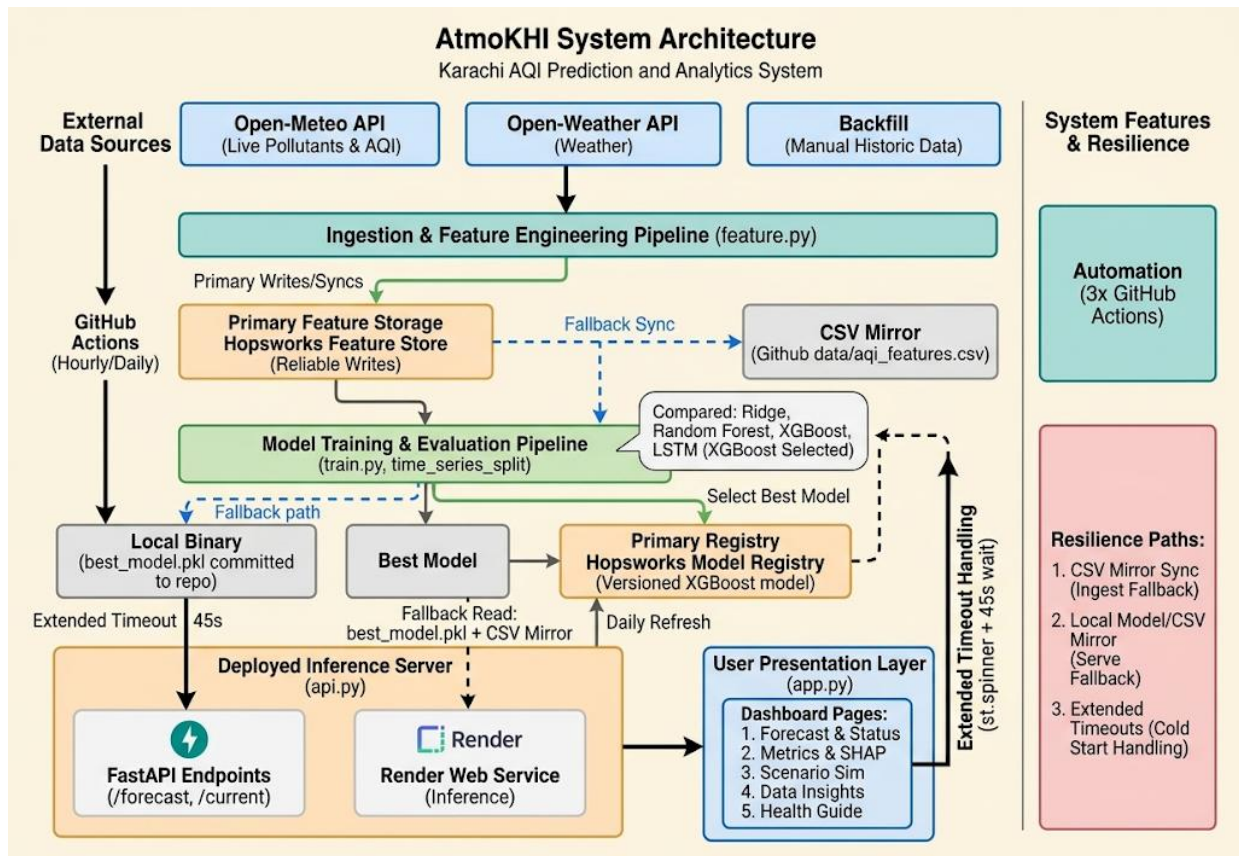


Figure 1: End to end architecture showing the feature pipeline, training pipeline, model registry, and web application, along with the fallback paths used when Hopworks reads were unreliable (credit limit exceeded).

1.2 What Each Component Does

- Feature pipeline:** Fetches live pollutant ($PM_{2.5}$, PM_{10} , O_3 , NO_2 , CO , SO_2) and weather data hourly, builds time/lag features, and writes to the Hopworks feature store with a CSV backup mirror on GitHub.
- Training pipeline:** Reads stored features daily, trains and compares four model architectures using time-aware cross-validation, and registers the best-performing model.
- Web application:** A FastAPI backend serves live predictions and recursive 72-hour forecasts, while a Streamlit dashboard presents interactive visualization tabs.

- **Automation layer:** Three GitHub Actions workflows keep feature ingestion, backfill, and retraining running automatically on schedule.

2. Automated Data and Training Pipeline

Three GitHub Actions workflows automate system maintenance without manual intervention:

Feature Pipeline (hourly): Ingests pollutant readings from Open-Meteo and weather data from OpenWeather, builds features, writes to Hopsworks, and appends to the GitHub CSV mirror to ensure read reliability.

Backfill Pipeline (one time / manual): Ingested ~12 months of historical hourly data from Open-Meteo to populate baseline lag and rolling window features for training.

Model Training Pipeline (daily): Retrains all models daily using time-aware cross-validation, evaluates the top model on held-out data, updates SHAP explainability plots, and saves model binaries to the Hopsworks registry and repository.

456 workflow runs		Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
🟡	Feature Pipeline - Hourly AQI Fetch Feature Pipeline - Hourly AQI Fetch #477: Manually run by murtaza-23	main		🕒 8 minutes ago 🔄 In progress		...
🟢	Daily AQI Model Training Daily AQI Model Training #45: Manually run by murtaza-23	main		🕒 Today at 2:34 PM 🕒 15m 0s		...
🟢	Feature Pipeline - Hourly AQI Fetch Feature Pipeline - Hourly AQI Fetch #476: Scheduled	main		🕒 Today at 1:41 PM 🕒 26m 19s		...
🟢	Feature Pipeline - Hourly AQI Fetch Feature Pipeline - Hourly AQI Fetch #475: Scheduled	main		🕒 Today at 7:28 AM 🕒 20m 14s		...
🟢	Feature Pipeline - Hourly AQI Fetch Feature Pipeline - Hourly AQI Fetch #474: Scheduled	main		🕒 Today at 4:42 AM 🕒 20m 53s		...
🟢	Feature Pipeline - Hourly AQI Fetch Feature Pipeline - Hourly AQI Fetch #473: Scheduled	main		🕒 Today at 2:23 AM 🕒 33m 19s		...

3. Feature Engineering and Exploratory Analysis

Before modelling, I explored the historical Karachi AQI data to understand its main patterns. The full analysis and plots are in the notebooks folder.

Key EDA findings

- AQI consistently peaks during morning (08:00–10:00) and evening (17:00–20:00) rush hours, supporting hour-of-day features.
- Weekday pollution is noticeably higher than weekend levels, supporting the "is_weekend" feature.

- Cooler winter months trap pollutants closer to ground level due to temperature inversion.
- $PM_{2.5}$ exhibited the strongest linear correlation with target AQI among all six tracked pollutants.

Final 32-Feature Set:

Pollutant concentrations ($PM_{2.5}$, PM_{10} , O_3 , NO_2 , CO , SO_2), temporal markers (hour, day, day_of_week, month, is_weekend, hour_category), and autoregressive AQI metrics (lags at 1, 3, 24, 48, 72 hours; 3h and 24h rolling averages; hour-over-hour change rate). Weather features such as temperature and humidity were excluded from training because historical weather data was not available on the free tier of the APIs used, so including it would have meant training on mostly missing values.



4. Model Training and Evaluation

The target variable is AQI one hour ahead ($t + 1$). Four algorithms were benchmarked: Ridge Regression, Random Forest, XGBoost, and a PyTorch LSTM.

4.1 Preventing Data Leakage & Model Selection

Random train-test splits cause temporal leakage. To prevent this, evaluation strictly utilized TimeSeriesSplit (5 folds), ensuring models were evaluated exclusively on unseen future time horizons. A naive Persistence Baseline ($AQI_{t+1} = AQI_t$) served as the benchmark.

Mean cross validated results across the model selection folds:

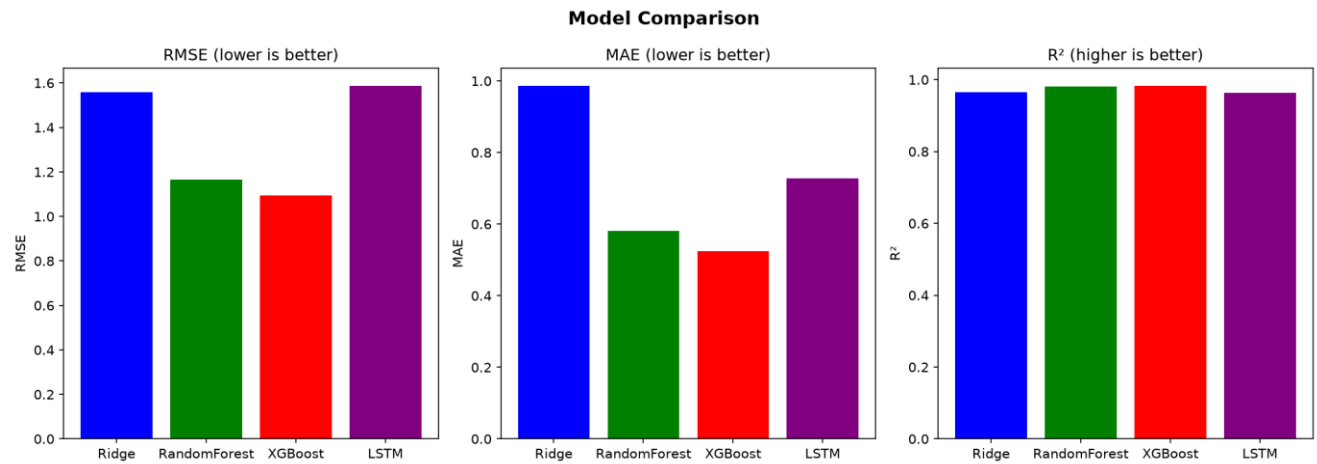
Model	RMSE	MAE	R ²
Ridge Regression	4.0377	2.4769	0.9491
Random Forest	3.8841	2.0413	0.9537
XGBoost (selected)	3.5129	1.8476	0.9628
PyTorch LSTM	1.58670	0.7277	0.9638

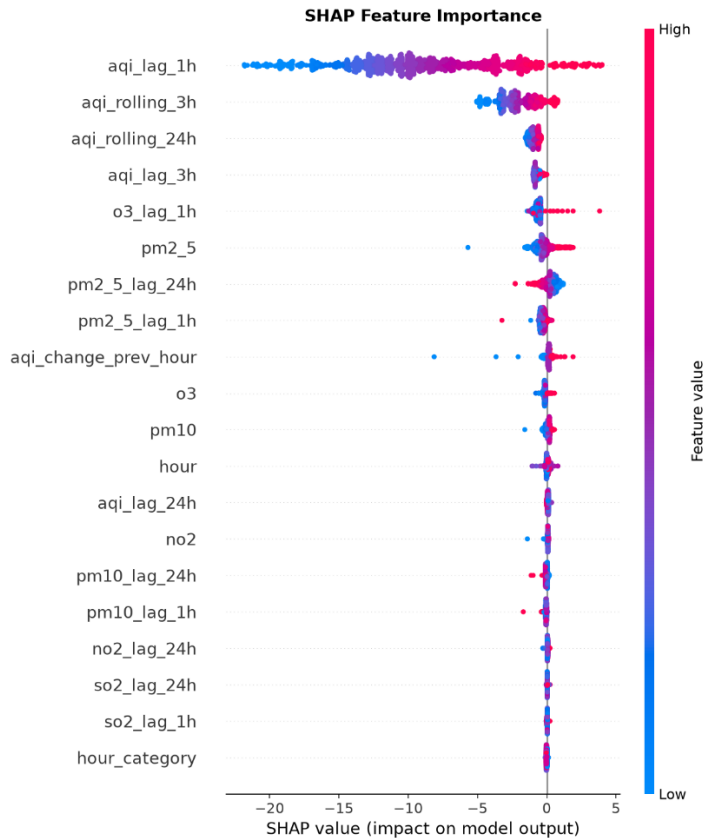
Final result on the fully held out fold, after selecting **XGBoost** as the best model:

Metric	XGBoost	Persistence	Improvement
RMSE (held-out fold)	1.0935	1.2917	15.3%
MAE (held-out fold)	0.5244	n/a	n/a
R ² (held-out fold)	0.9828	n/a	n/a

Note: The validation metrics above reflect performance at the time of initial evaluation; as the daily automated pipeline ingests new atmospheric data and retrains, these values will naturally adjust over time.

XGBoost was selected for production because it trained faster, deployed without heavy deep-learning runtime overhead, and yielded stable predictions on tabular lag features.





4.2 SHAP explainability

SHAP was used to understand which features the model actually relies on. As expected, the AQI lag features, particularly the previous hour value, dominate the model's predictions, followed by PM_{2.5}. This confirms that the model is behaving sensibly rather than picking up on noise.

4.3 Multi horizon evaluation

The web app forecasts 72 hours ahead, but the model itself is only trained to predict one hour ahead. To understand how well a single hour ahead model performs when applied further into the future, I ran a separate evaluation where the model is applied directly against targets 24, 48 and 72 hours away, without recursively chaining predictions.

Forecast Horizon	Model RMSE	Persistence RMSE	R ²
24 hours	16.0233	16.2835	0.4558
48 hours	20.4848	20.5415	0.1128
72 hours	22.669	22.7188	-0.0891

As expected, accuracy drops the further ahead we try to predict, which is normal behaviour for this type of forecasting problem and is consistent with how autocorrelated a signal like AQI is. It is worth being clear that this table measures something slightly different from the live 72-hour forecast shown in the dashboard, which instead uses recursive forecasting explained below.

4.4 Recursive 72-hour forecasting

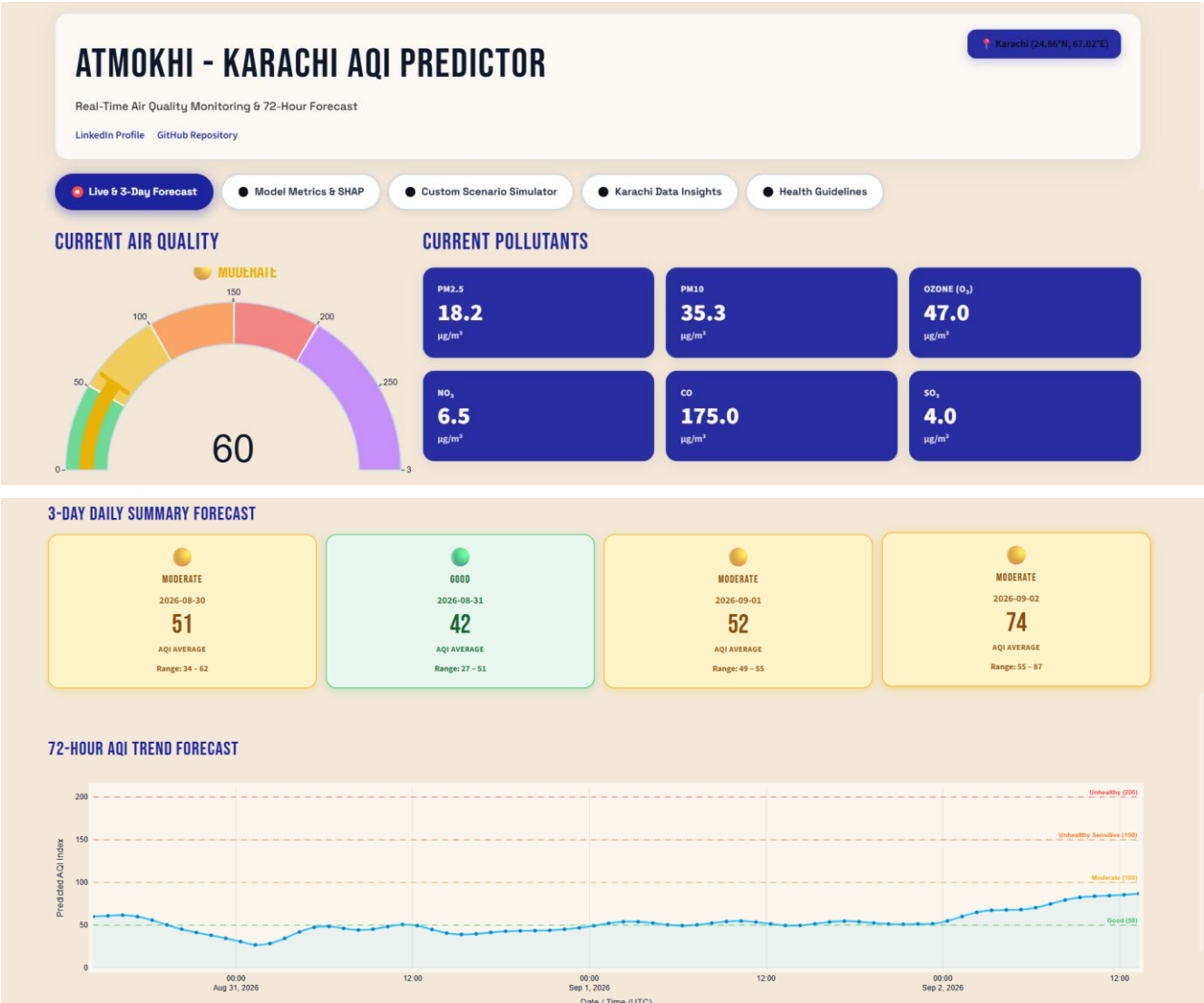
Since the trained model only predicts one hour ahead, the dashboard builds a full 3 day forecast by looping the model over itself. It predicts hour $t + 1$, then feeds that prediction back in as the new lag feature to predict $t + 2$ and repeats this all the way to $t + 72$. Hour of day pollutant averages from historical data are used to keep the forecast realistic instead of freezing pollutant levels at whatever they were at the start of the forecast.

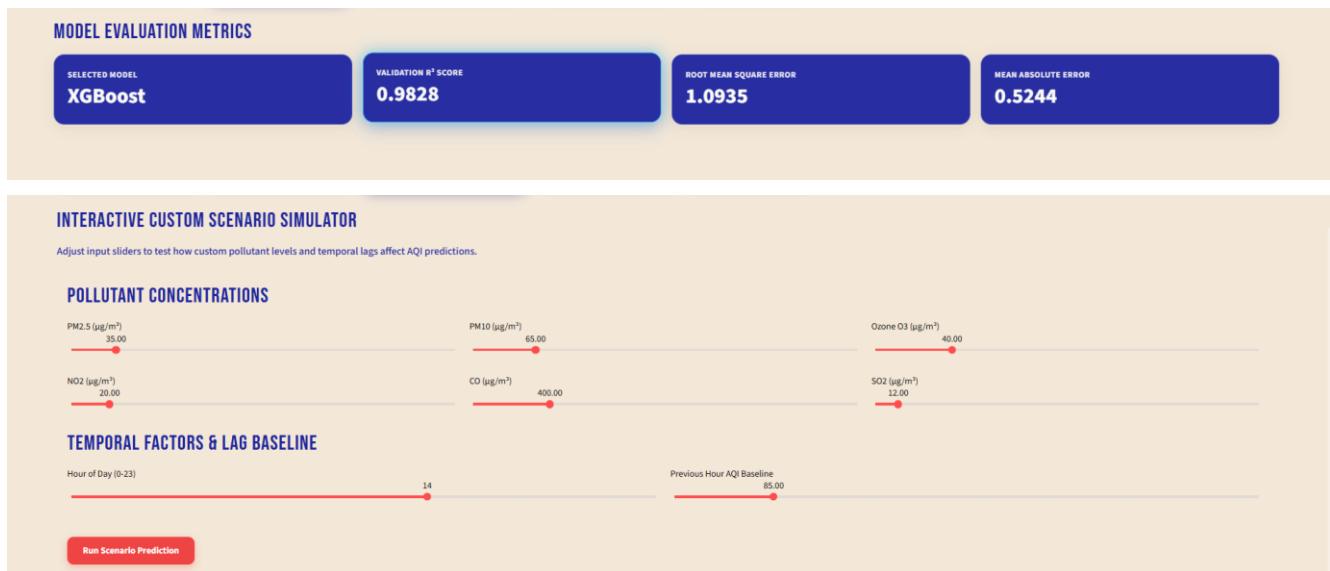
5. Web Application

The system is deployed via a FastAPI backend on Render (serving prediction endpoints) and a Streamlit dashboard on Streamlit Cloud providing 5 interactive tabs:

1. **Live & 3-Day Forecast:** Current AQI gauge, live pollutants, and 72-hour forecast chart.
2. **Model Metrics & SHAP:** Cross-validation scores and feature importance plots.
3. **Custom Scenario Simulator:** Interactive input sliders to test custom pollutant scenarios.
4. **Karachi Data Insights:** Charts covering daily, weekly, and seasonal patterns.
5. **Health Guidelines:** Precautionary advice mapped to EPA AQI categories.

Hazard Alerts: Automated banners notify users whenever current or predicted AQI crosses into unhealthy thresholds (AQI > 150).





6. Problems Faced and How I Solved Them

The project also involved several practical engineering problems. Solving these issues was an important part of the work.

6.1 Static Data Source: Initial ground-station API (AQICN) updated infrequently. Switched to Open-Meteo for continuous hourly pollutant modeling.

6.2 Hopsworks Read Failures and credit limits: Query socket timeouts interrupted reads. Implemented an automated dual write to a synchronized GitHub CSV mirror used for training and serving fallbacks.

6.3 Temporal Data Leakage: Random splits cause leakage. Enforced strict TimeSeriesSplit cross-validation and evaluated against persistence baselines.

6.4 Deployment Runtime Mismatch: Render defaulted to Python 3.14, breaking dependencies. Pinned deployment environment to Python 3.11 via runtime.txt.

6.5 Multi-Day Forecast Generation: Model only predicts $t + 1$ ahead. Built an autoregressive recursive loop feeding predicted values back as lag features out to 72 hours.

7. Conclusion and What Could Be Improved

AtmoKHI covers the full ML lifecycle: automated data collection, feature engineering, model comparison, time-aware evaluation, explainability, and deployment. The main workflows run automatically on schedule.

Future enhancements include integrating paid historical weather tiers, adding prediction confidence intervals, and extending pipeline coverage to other major urban centers across Pakistan.